PERTEMUAN 5 (KUNCI JAWABAN)

1. Dalam pembuatan REST API, kita sering menggunakan method GET , POST , PUT, dan DELETE. Jelaskan perbedaan keempat method tersebut ! (5 poin)

- GET berguna untuk mengambil / mendapatkan data
- POST berguna untuk mengirimkan data ke sisi server dalam bentuk request yang biasanya berupa JSON.
- PUT berguna untuk memodifikasi data yang sudah ada.
- DELETE berguna untuk menghapus data

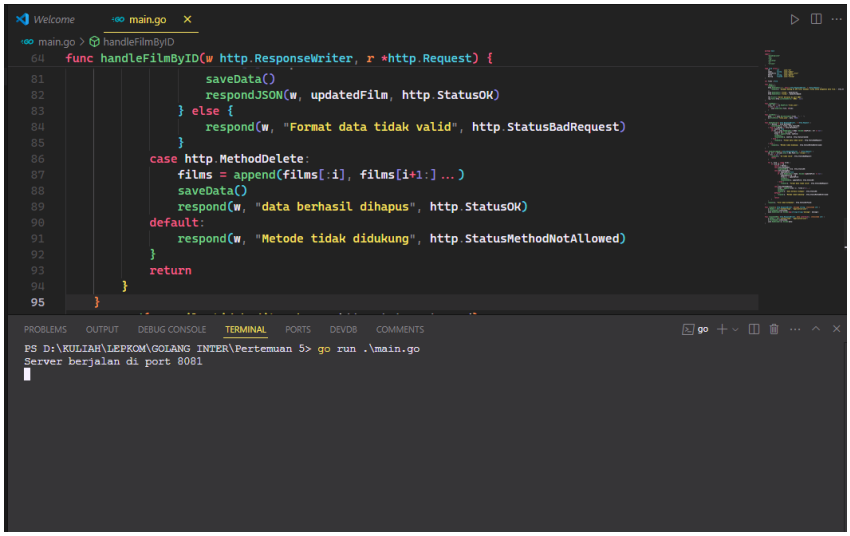
2. Perhatikan potongan code berikut :

```go
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello World!")
})
```

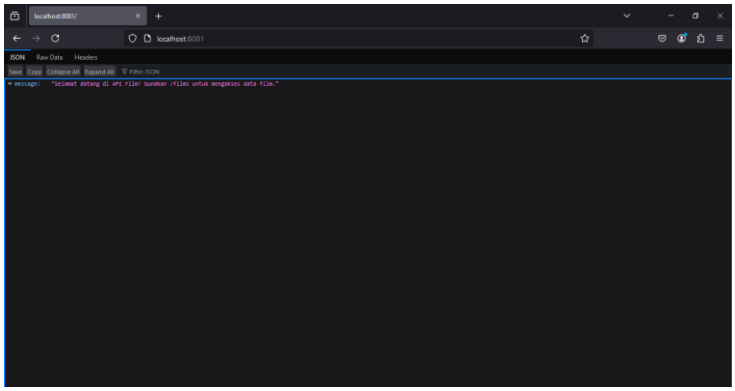
Jelaskan secara singkat cara kerja dari potongan kode diatas ! (5 poin)

Perintah code di atas adalah perintah apabila kita mengakses endpoint default atau endpoint "/" akan mengeluarkan atau menampilkan "Hello World".

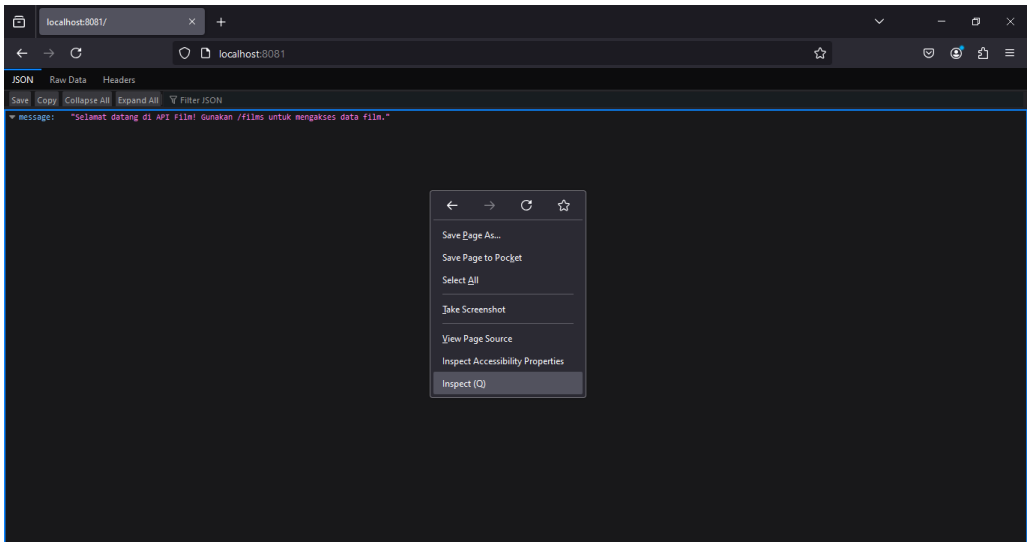
CARA MENJALANKAN API DENGAN SEMUA METHOD DI MOZILLA FIREFOX (DIKARENAKAN POSTMAN TERLALU BERAT UNTUK PC TERCINTA)

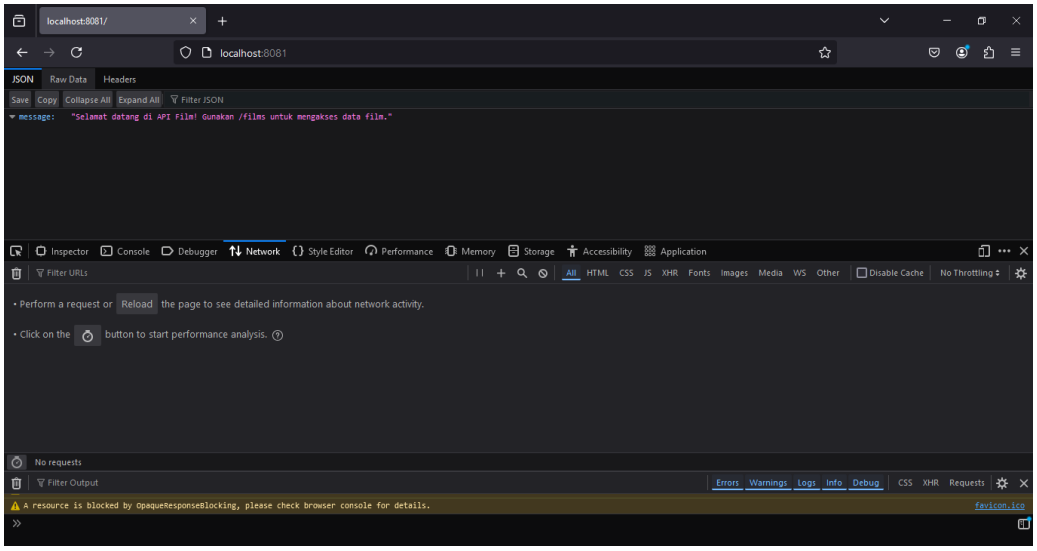
- 

  Pastikan untuk run golang terlebih dahulu.

- 

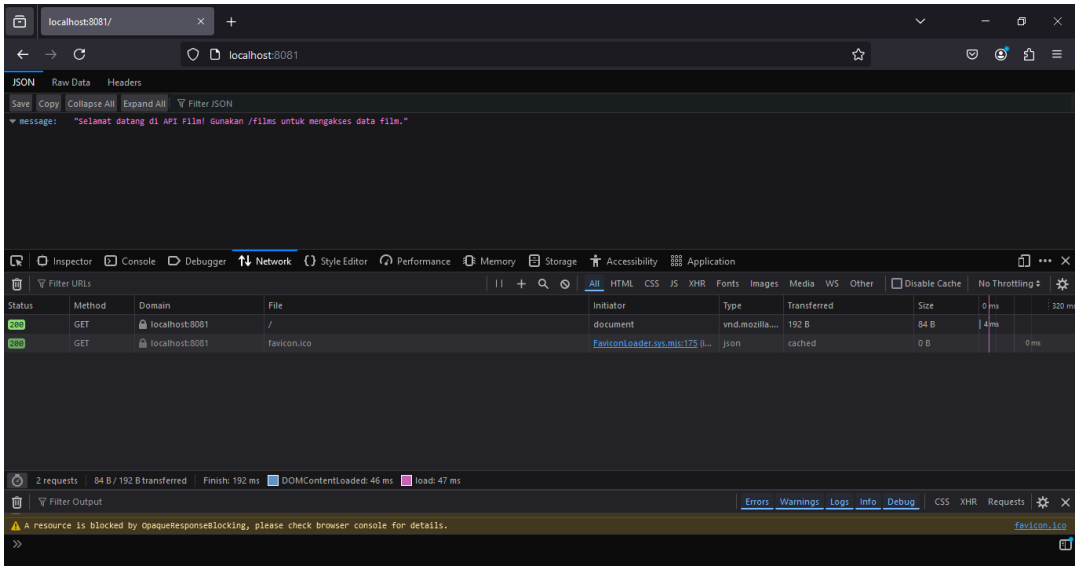
Bukalah firefox dan akses ke url localhost dengan port yang dibuat pada hardcode seperti disini contohnya adalah 8081 maka urlnya adalah localhost:8081/

- 

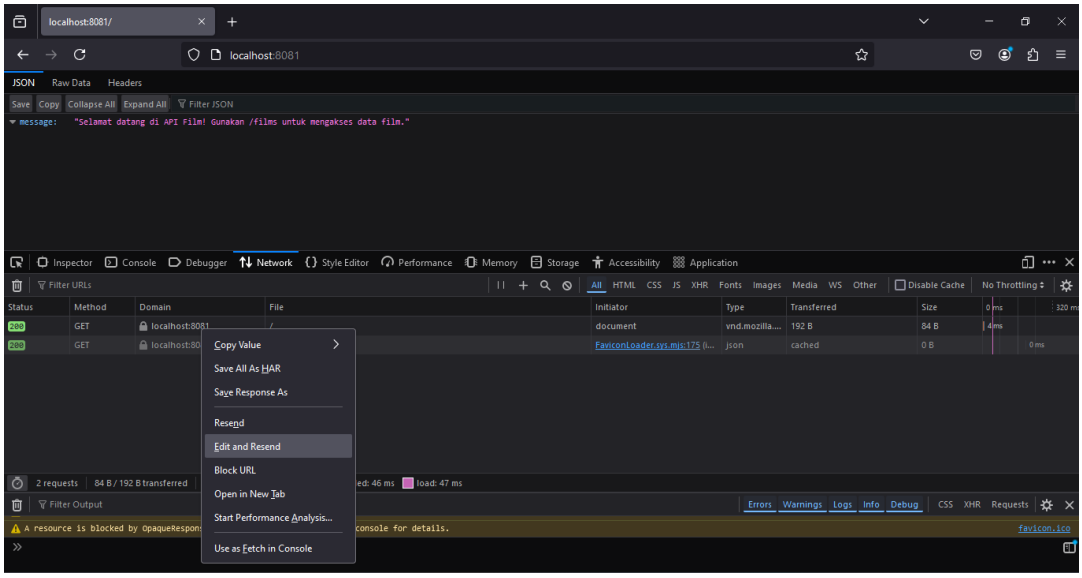


Lalu klik kanan pada firefox dan klik Inspect lalu ke bagian Network sampai seperti di gambar. Kemudian klik refresh

-

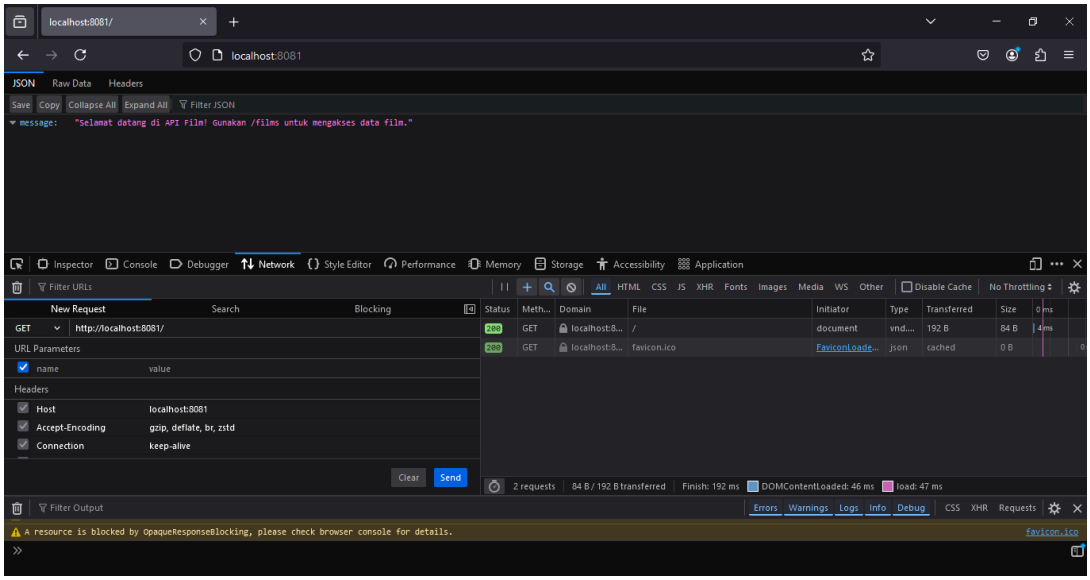


-

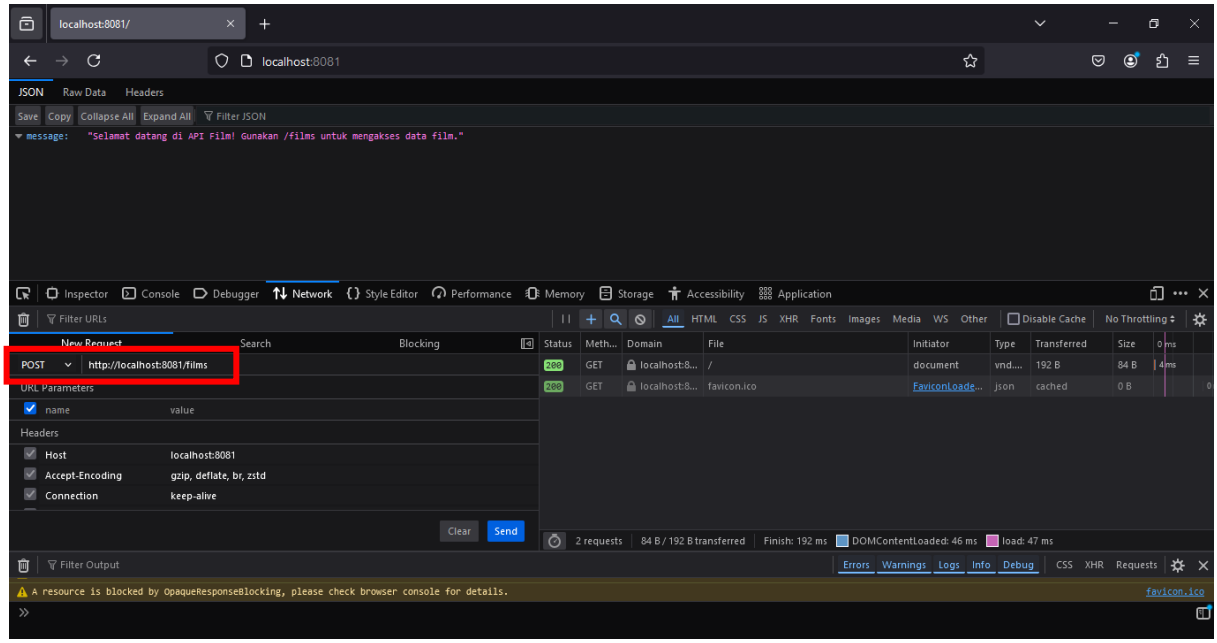


lalu klik kanan di domain localhost yang File atau endpointnya "/" dan pilih Edit and Resend.

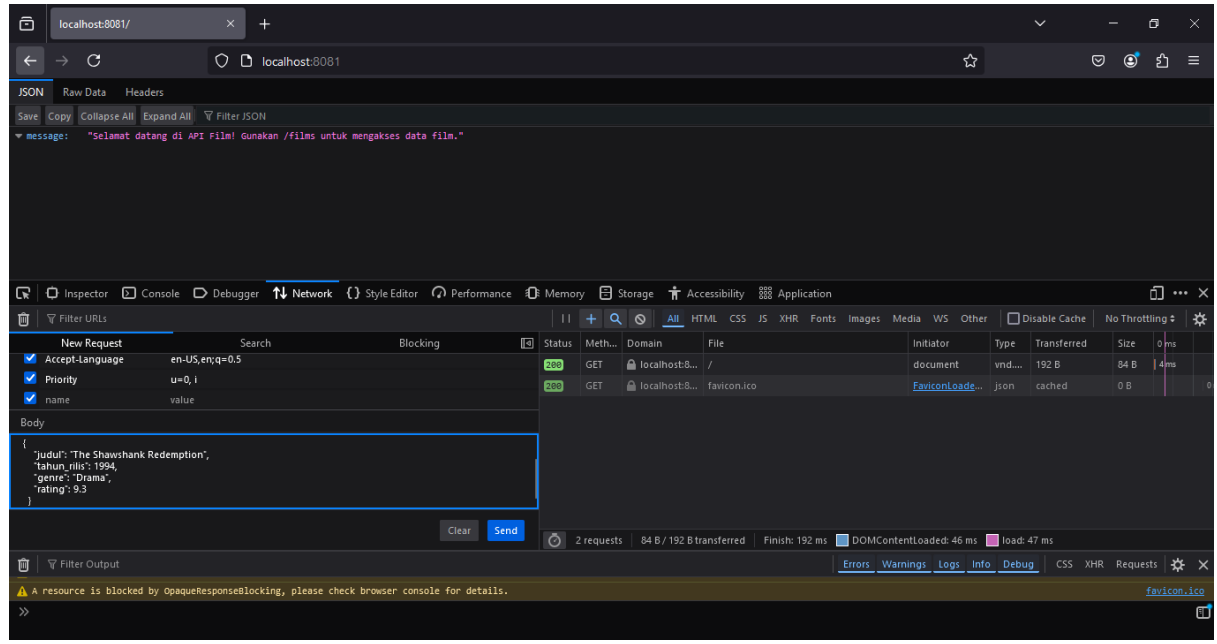
-



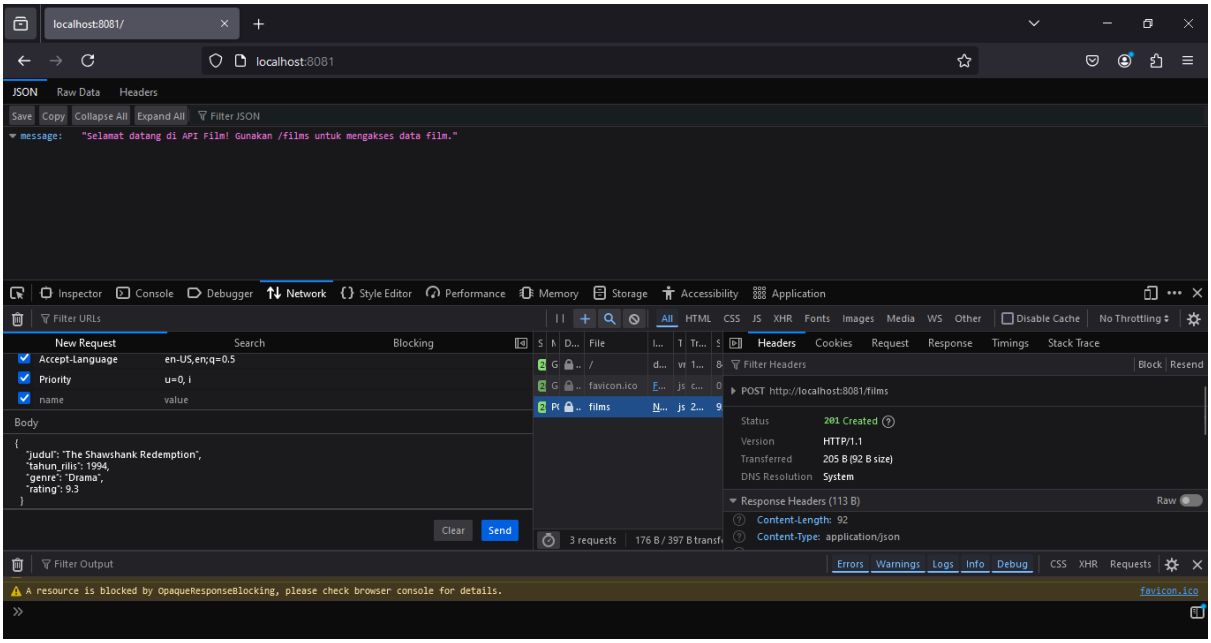
Lalu akan muncul tab seperti ini dan kalian bisa mengubah methodnya menjadi **POST**, **PUT**, maupun **DELETE**. Cara seperti ini digunakan untuk mengirim request ke server dengan method tiga tersebut.

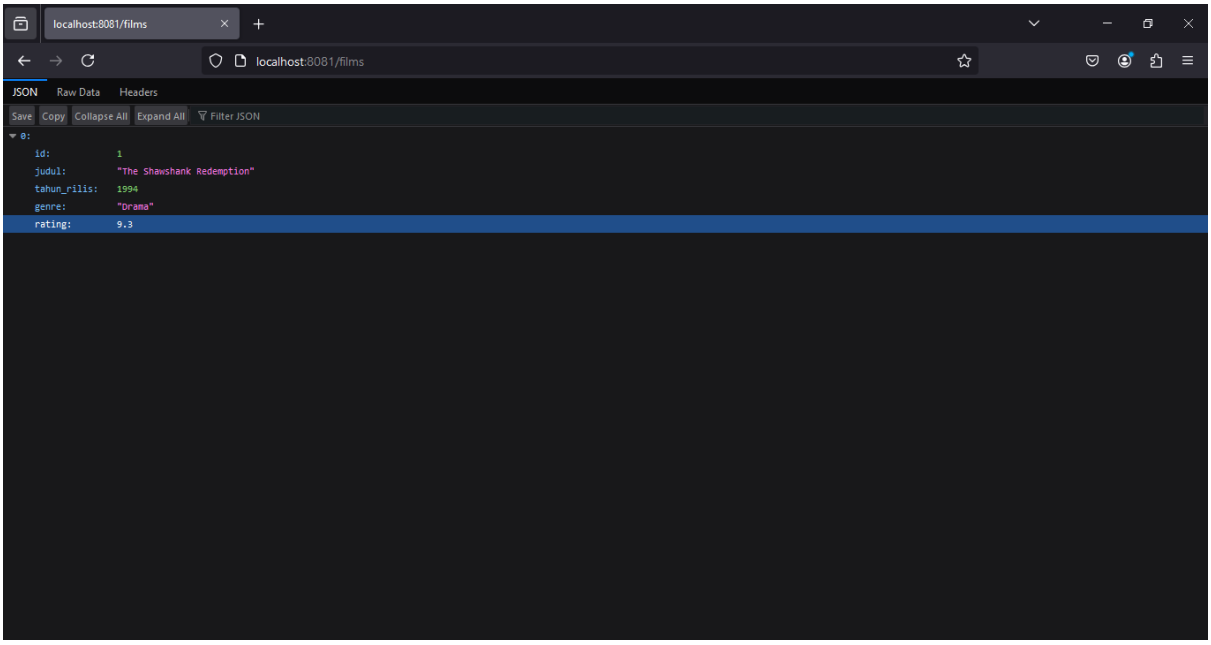


-



Berikut adalah contoh penggunaan method POST di Firefox. Setelah mengganti method menjadi POST, discroll ke bawah aja nanti ada request bodynya. Setelah itu tinggal "Send" aja nanti hasilnya seperti ini :


localhost:8081/
localhost:8081
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
message: "Selamat datang di API Film! Gunakan /films untuk mengakses data film."
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Filter URLs
All
HTML
CSS
JS
XHR
Fonts
Images
Media
WS
Other
Disable Cache
No Throttling
New Request
Search
Blocking
Accept-Language
en-US,en;q=0.5
Priority
u=0, i
name
value
Body
{
"judul": "The Shawshank Redemption",
"tahun_rilis": 1994,
"genre": "Drama",
"rating": 9.3
}
Clear
Send
favicon.ico
films
Headers
Cookies
Request
Response
Timings
Stack Trace
Filter Headers
Block
Resend
POST http://localhost:8081/films
Status
201 Created
Version
HTTP/1.1
Transferred
205 B (92 B size)
DNS Resolution
System
Response Headers (113 B)
Raw
Content-Length: 92
Content-Type: application/json
3 requests
Filter Output
Errors
Warnings
Logs
Info
Debug
CSS
XHR
Requests
A resource is blocked by OpaqueResponseBlocking, please check browser console for details.
favicon.ico


localhost:8081/films
localhost:8081/films
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
0:
id: 1
judul: "The Shawshank Redemption"
tahun_rilis: 1994
genre: "Drama"
rating: 9.3

JAWABAN!

1. Importkan Library yang diperlukan ! (5 poin)

```go
main.go
main.go > handleFilmByID
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "strconv"
)
```

```
import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "strconv"
)
```

2. Buatkan struct film untuk menyimpan data sesuai struktur yang diminta, Struktur yang diharapkan (5 poin) :

```go
Codeium: Refactor | Explain
type Film struct {
    ID          int     `json:"id"`
    Judul       string  `json:"judul"`
    TahunRilis  int     `json:"tahun_rilis"`
    Genre       string  `json:"genre"`
    Rating      float64 `json:"rating"`
}

var films []Film
```

```
type Film struct {
    ID      int     `json:"id"`
```

```
        Judul      string  `json:"judul"`
        TahunRilis int     `json:"tahun_rilis"`
        Genre      string  `json:"genre"`
        Rating     float64 `json:"rating"`
    }
```

Seperti biasa disimpan dalam bentuk struct dengan nama struct nya adalah Film dengan data – data seperti yang diminta pada soal act yaitu ID dengan tipe data integer dan akan dibaca oleh json dengan id, Judul dengan tipe data string dan json akan membaca dengan judul, dan seterusnya.

3. Buatlah variable films bertipe array dengan struct film (5 poin)

```go
var films []Film
```

```
var films []Film
```

Lalu struct tersebut akan disimpan di variable bertipe array bernama films.

4. Buatlah sistem untuk membaca dan menulis ke file JSON dengan spesifikasi berikut (5 poin) :
   a. Fungsi loadData()
      - Membaca file films.json
      - Unmarshal data ke variable films
      - Handle error jika fila tidak ditemukan
   b. Fungsi saveData
      - Marshal data films ke format JSON
      - Simpan ke file films.json
      - Handle error saat proses penyimpanan

a.

```go
Codeium: Refactor | Explain | Generate GoDoc | X | Qodo Gen: Options | Test this function
func loadData() {
    file, err := os.ReadFile("films.json")
    if err == nil {
        json.Unmarshal(file, &films)
    }
}
```

```
func loadData() {
        file, err := os.ReadFile("films.json")
        if err == nil {
                json.Unmarshal(file, &films)
        }
}
```

Dibuat function dengan nama loadData yang mana function ini berfungsi untuk membaca file dari file yang bernama films.json, jadi apabila belum membuat file films.json bisa dibuat terlebih dahulu jika nantinya akan menjalankan perintah GET terlebih dahulu. Lalu digunakanlah perkondisian kalau tidak ditemukan error maka package “encoding/json” akan menjalankan function Unmarshal yang berguna untuk mengubah string JSON menjadi struktur data.

File films.json (untuk judul film bisa diubah sesuai keinginan, ini hanya template):

```
main.go   ×   {} films.json ×
{} films.json > ...
1  [
2      {
3        "id": 1,
4        "judul": "The Shawshank Redemption",
5        "tahun_rilis": 1994,
6        "genre": "Drama",
7        "rating": 9.3
8      },
9      {
10       "id": 2,
11       "judul": "The Godfather",
12       "tahun_rilis": 1972,
13       "genre": "Crime",
14       "rating": 9.2
15     },
16     {
17       "id": 3,
18       "judul": "Inception",
19       "tahun_rilis": 2010,
20       "genre": "Sci-Fi",
21       "rating": 9
22     }
23   ]
```

```
[
  {
   "id": 1,
```

```
    "judul": "The Shawshank Redemption",
    "tahun_rilis": 1994,
    "genre": "Drama",
    "rating": 9.3
  },
  {
    "id": 2,
    "judul": "The Godfather",
    "tahun_rilis": 1972,
    "genre": "Crime",
    "rating": 9.2
  },
  {
    "id": 3,
    "judul": "Inception",
    "tahun_rilis": 2010,
    "genre": "Sci-Fi",
    "rating": 9
  }
]
```

b.

```
Codeium: Refactor | Explain | Generate GoDoc | X | Qodo Gen: Options | Test this function
func saveData() {
    data, _ := json.MarshalIndent(films, "", "  ")
    os.WriteFile("films.json", data, 0644)
}
```

```
func saveData() {
        data, _ := json.MarshalIndent(films, "", "  ")
        os.WriteFile("films.json", data, 0644)
}
```

Selanjutnya buat function bernama saveData yang mana function ini berfungsi untuk menyimpan di variable data dengan catatan bahwa json.MarshalIndent akan mengonversi data pada variable films menjadi format JSON. Fungsi os.WriteFile akan membuat file baru bernama films.json, jadi kalau tutor tidak ingin membuat file manual

dan membuat data manual maka tutor bisa menjalankan method POST terlebih dahulu karena kalau menjalankan perintah GET maka datanya akan kosong. Argumen ketiga yaitu 0644 pada fungsi os.WriteFile adalah untuk meminta izin file yang mana file bisa dibaca oleh semua pengguna, tetapi hanya bisa ditulis oleh pemiliknya.

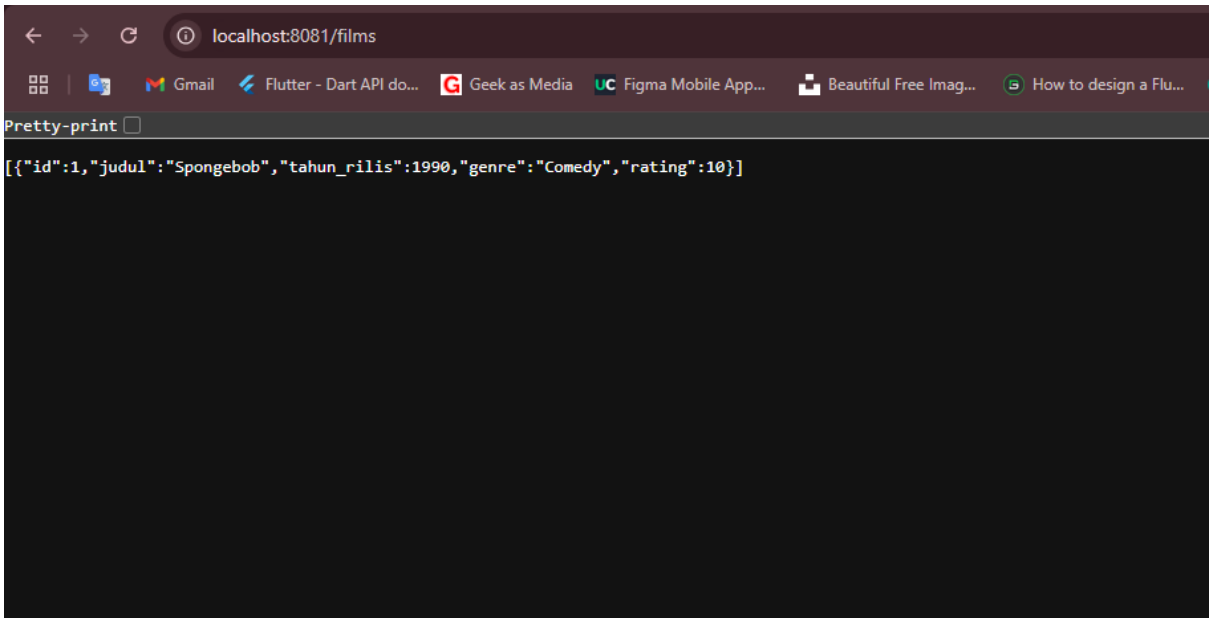
5. Buatlah helper function untuk memformat response dalam bentuk string ataupun JSON (10 poin) seperti code dibawah

```
func respond(w http.ResponseWriter, message string, statusCode int) {
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(statusCode)
        json.NewEncoder(w).Encode(map[string]string{"message":
message})
}

func respondJSON(w http.ResponseWriter, data interface{}, statusCode int)
{
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(statusCode)
        json.NewEncoder(w).Encode(data)
}
```

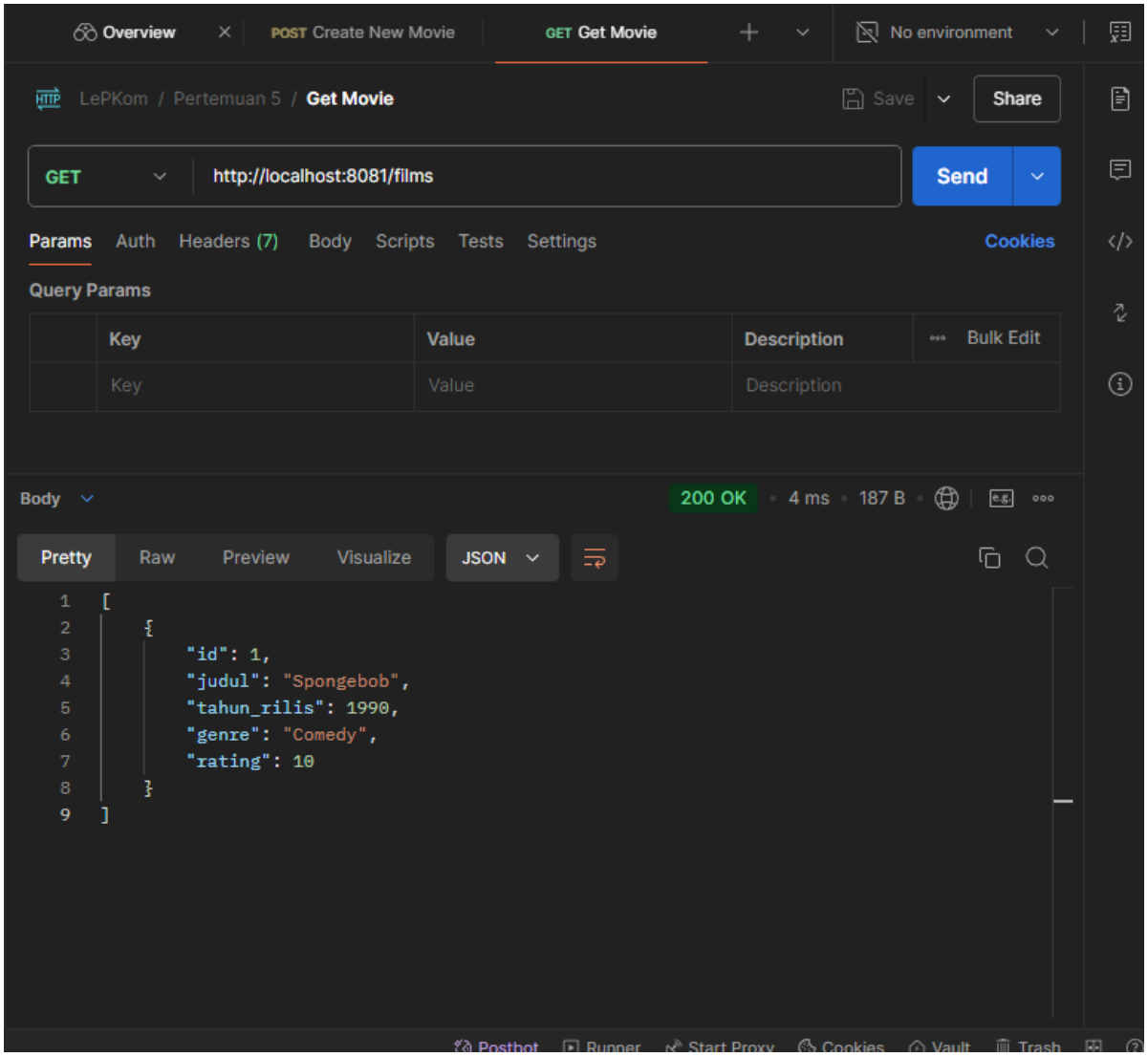
```
func main() {
   loadData()
   http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        respond(w, "Selamat datang di API Film! Gunakan /films untuk
mengakses data film.", http.StatusOK)
   })
   http.HandleFunc("/films", handleFilms)
   http.HandleFunc("/films/", handleFilmByID)

   fmt.Println("Server berjalan di port 8081")
   log.Fatal(http.ListenAndServe(":8081", nil))
}
```

6. Buatlah endpoint dengan method GET dengan route “/films” untuk menampilkan semua film (5 poin)

   Buat soal ini bisa langsung buka chrome aja atau buka postman nanti hasilnya bakalan kayak gini:



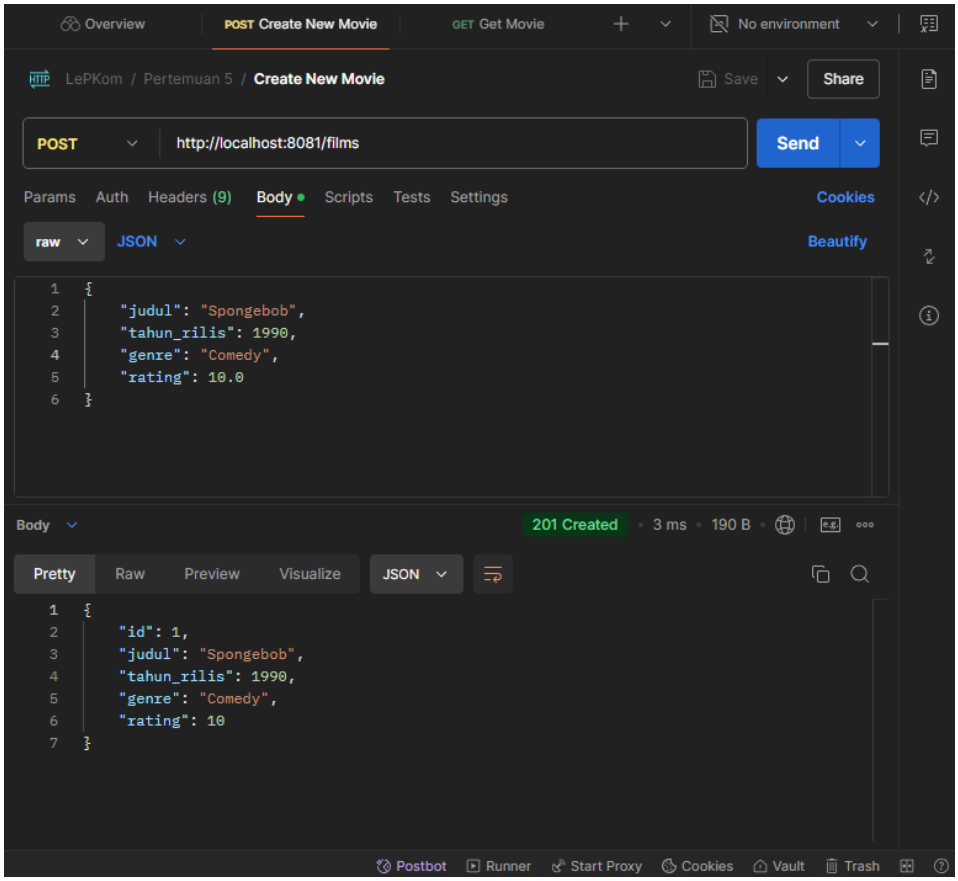
   Atau



7. Buatlah endpoint dengan method POST dengan route “/films” untuk menambahkan film baru dengan ID auto-increment (10 poin)

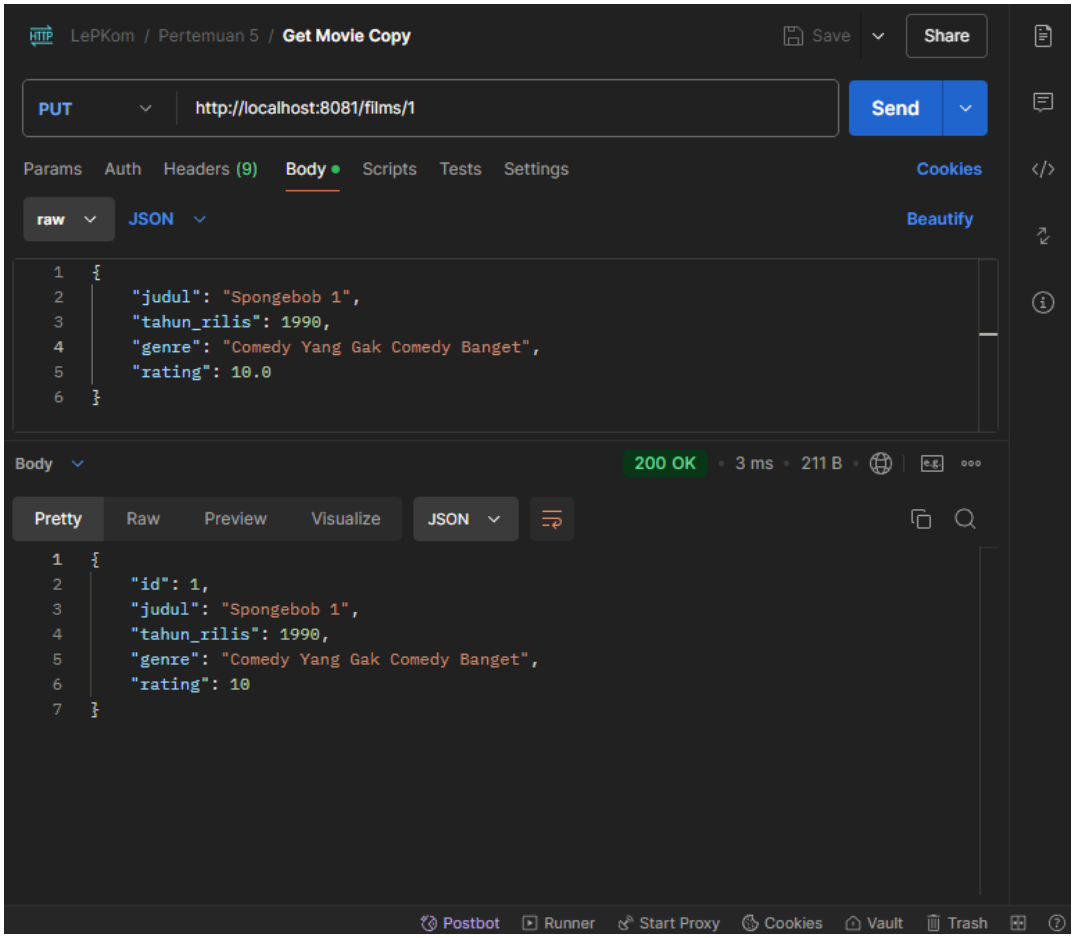
   Karena ID nya adalah auto increment jadi otomatis nambah sendiri, jadi cukup buat request body di Postman seperti berikut :

   Untuk 1 data :



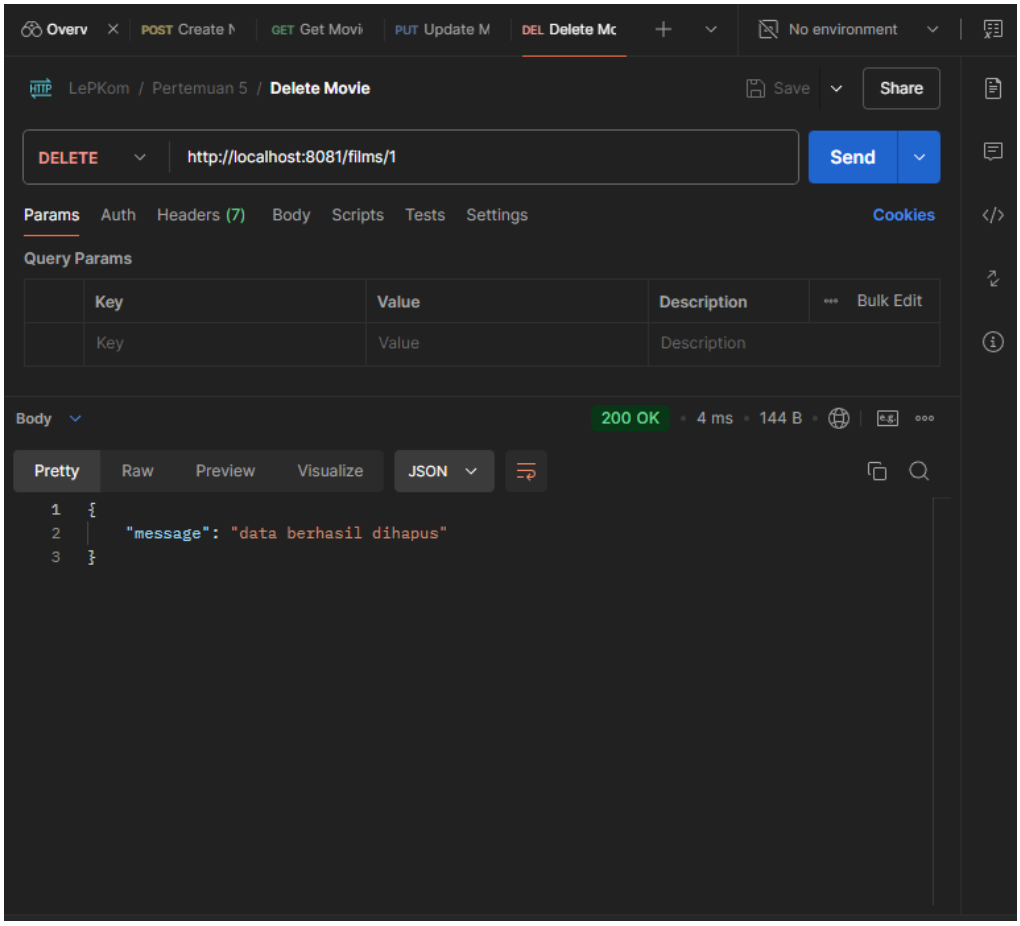

```
{
    "judul": "Spongebob",
    "tahun_rilis": 1990,
    "genre": "Comedy",
    "rating": 10.0
}
```

8. Buatlah endpoint dengan method PUT dengan route “/films/{id}” untuk mengupdate informasi film berdasarkan ID (10 poin):



Untuk Request body nya samain aja kayak pas mau bikin data baru tapi methodnya diubah jadi PUT karena buat modifikasi data. Terus di endpointnya tambahin id di paling belakang, contohnya disini id nya adalah 1.

9. Buatlah endpoint dengan method DELETE dengan route "/films/{id}" untuk menghapus film berdasarkan ID (10 poin)



Untuk method DELETE tidak memerlukan request body jadi cukup ganti method PUT ke DELETE saja karena pada soal sebelumnya sudah menggunakan id untuk memodifkasi datanya.

10. Buatlah Error Handling & Response format untuk penanganan scenario (25 poin) :
    - Film tidak ditemukan
    - Format request tidak valid
    - ID tidak valid

Buat nomor 10 ada dua function ini ya namanya **handleFilms** sama **handleFilmByID**:

```
func handleFilms(w http.ResponseWriter, r *http.Request) {
    if r.Method == http.MethodGet {
        respondJSON(w, films, http.StatusOK)
    } else if r.Method == http.MethodPost {
        var newFilm Film
        if err := json.NewDecoder(r.Body).Decode(&newFilm); err == nil {
```

```
            newFilm.ID = len(films) + 1
            films = append(films, newFilm)
            saveData()
            respondJSON(w, newFilm, http.StatusCreated)
        } else {
            respond(w, "Format data tidak valid", http.StatusBadRequest)
        }
    } else {
        respond(w, "Metode tidak didukung", http.StatusMethodNotAllowed)
    }
}

func handleFilmByID(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.URL.Path[len("/films/"):])
    if err != nil {
        respond(w, "ID tidak valid", http.StatusBadRequest)
        return
    }

    for i, film := range films {
        if film.ID == id {
            switch r.Method {
            case http.MethodGet:
                respondJSON(w, film, http.StatusOK)
            case http.MethodPut:
                var updatedFilm Film
                if json.NewDecoder(r.Body).Decode(&updatedFilm) == nil {
                    updatedFilm.ID = id
                    films[i] = updatedFilm
                    saveData()
                    respondJSON(w, updatedFilm, http.StatusOK)
                } else {
                    respond(w, "Format data tidak valid", http.StatusBadRequest)
```

```
            }
        case http.MethodDelete:
            films = append(films[:i], films[i+1:]...)
            saveData()
            respond(w, "data berhasil dihapus", http.StatusOK)
        default:
            respond(w, "Metode tidak didukung", http.StatusMethodNotAllowed)
        }
        return
    }
  }
  respond(w, "Film tidak ditemukan", http.StatusNotFound)
}
```

Good luck para tutor, semoga kj ini membantu yaaw <3